

# Neural Branching and Shared Feature Space

By Arkadyuti Maiti

## 1. Abstract

This project explores a novel network architecture for neural networks based on **feature-space branching** where a new “child” neural network is branched from a particular hidden layer of the parent network. This allows a particular child branch to achieve greater specialization of certain classes at a lower compute cost.

The system is implemented using a custom lightweight framework and evaluated on a filtered subset of the **CIFAR-100** dataset, focusing on hierarchical class structures such as flowers and aquatic animals. Branch networks are trained on activations extracted from hidden layers of the parent network, enabling specialization without explicit routing mechanisms.

## 2. Introduction

Modern neural networks have achieved great success across various fields, particularly in classification and regression. These networks often follow a monolithic architecture where a single network learns to map inputs directly to outputs through end-to-end training. However these networks struggle at hierarchical or multi-level classification problems.

In real life, data often shows a hierarchical pattern within itself. For example, objects can first be categorized into coarse groups (such as animals vs. plants) and then further divided into finer subclasses (such as species). Traditional neural networks will activate and use the whole model to just detect a target subclass which is highly inefficient and unspecialized.

This project introduces the concept of **neural branching**, a framework in which specialized subnetworks, or *branches*, emerge from intermediate layers of a parent network. The parent model learns a shared feature representation of the input data, while branch networks operate on this feature space to perform more specific classification tasks. This design promotes **feature reuse**, **modularity**, and **task specialization**, while maintaining a unified learning pipeline.

## 4. Torchly – A Pytorch wrapper

Before starting this project, I felt the need for an AI development library that allows rapid prototyping and deployment without the hassle of creating long unnecessary classes and objects. Unfortunately, Pytorch, the library of choice for this project required creating classes and objects that could hinder the speed of my experiments.

So, I created **Torchly**, a wrapper over Pytorch library which allows me to rapidly prototype large and complex neural networks with minimal code while also giving me the flexibility to change model parameters like activation functions, learning rate, loss functions etc...

Torchly is also available on GitHub. [ <https://github.com/ciada-3301/Torchly> ]

## 5. CIFAR100 – Superclass and subclasses used

For this experiment, we have use 2 superclasses with 3 subclasses each. Namely:

- 1) **Flowers:** Sunflower, Poppy, Tulip.
  - 2) **Fishes:** Crab, Aquarium Fish, Shark
- (Normalized and Filtered data used only)

CIFAR-100 is perfect for this project as it offers the exact hierarchical system as demanded by the experiment with proper quality of data.

## 6. Approach/Methodology

### 6.1 Parent Network

The parent network serves as the foundational model responsible for learning a generalized representation of the input data. The architecture consists of a fully connected feedforward network with dimensions structured as **[3072, 1024, 512, 6]**, where the input size corresponds to flattened CIFAR-100 images ( $32 \times 32 \times 3 = 3072$ ). The network learns to classify between all 6 classes as discussed in 5.

### 6.2 Feature Extraction

Instead of relying solely on the final output layer, this approach leverages **hidden layer activations** from the parent network as a learned feature space. For this particular project I have decided to use the 2<sup>nd</sup> hidden layer of the parent (512 neurons). Lower

layers capture basic patterns (edges, textures) while Higher layers become too task-specific.

### 6.3 Branched Networks

Two branches emerge from the 2<sup>nd</sup> hidden layer of the parent network with the same architecture as: [512,256,3] and uses **Relu activation** with **cross\_validation** loss:

- (1) Fishnet: Specializes in classifying between crab, aquarium fish and shark.
- (2) Flownet: Specializes in classifying between sunflower, poppy and tulip.

## 7. Training

The complete training for this experiment is done in 3 stages.

### 7.1 Parent Training:

The parent network is trained on the selected dataset (coarse or multi-class) using standard supervised learning. The goal is to learn a robust and general feature representation.

### 7.2 Fish Branch Training:

Now we take a subset of our dataset containing only the images of fishes, pass it in the parent network, extract activations from the 2<sup>nd</sup> hidden layer. Then we pass the list of activations as input through our fishnet and use standard supervised learning to train the Fish Branch while the parent is frozen.

### 7.3 Flower Branch Training:

We take another subset of our dataset containing only the images of flowers, pass it in the parent network, extract activations from the 2<sup>nd</sup> hidden layer. Then we pass the list of activations as input through our fishnet and use standard supervised learning to train the Fish Branch while the parent is frozen. Same as fish branch training.

## 8. Outcomes and Results:

### 8.1 Parent Network:

During training, the parent network showed the following loss over 200 epochs using cross entropy:

| <b>Epochs</b> | <b>Loss</b> |
|---------------|-------------|
| 20/200        | 0.2894      |
| 40/200        | 0.1903      |
| 60/200        | 0.1140      |
| 80/200        | 0.1564      |
| 100/200       | 0.1114      |
| 120/200       | 0.1266      |
| 140/200       | 0.1080      |
| 160/200       | 0.1783      |
| 180/200       | 0.0832      |
| 200/200       | 0.1159      |

The accuracy of the parent does not matter as it is only being used for feature extraction.

## 8.2 Accuracy of Fishnet:

During testing the fishnet, the following metric were observed:

| <b>Metric</b>                   | <b>Value</b> |
|---------------------------------|--------------|
| <b>Total Test Samples</b>       | 300          |
| <b>Accepted Predictions</b>     | 250 (83.3%)  |
| <b>Overall Accuracy</b>         | 75.33%       |
| <b>Accuracy (Accepted Only)</b> | 80.00%       |

## 8.3 Accuracy on Flowernet:

During testing the fishnet, the following metric were observed: Generally, it underperformed compared to Fishnet due to low features in images at low resolution and the same blue hue in every picture that may have confused the network.

| Metric                   | Value       |
|--------------------------|-------------|
| Total Test Samples       | 300         |
| Accepted Predictions     | 224 (74.7%) |
| Overall Accuracy         | 67.33%      |
| Accuracy (Accepted Only) | 74.11%      |

## 9. Inference and Observations:

From the experiments, the following observations/inference:

1. A neural network can learn from the interpretation of another neural network. The new neural network does not necessarily need the output, but rather activations of a hidden layer as per requirements.
2. The branches in our case, fishnet and Flowernet, seemed to learn from the activations obtained from parent network.
3. Smaller neural networks that branch from the parent can specialize on certain features which can be useful in various use cases.

## 10. Some other useful information

1. The branched neural networks can further be branched into other branches for tasks that require even more specialization. For example: which kind of **diagnose cancer** -> **which kind?** -> **What stage?** or **What crop?** -> **what disease?** -> **localization**.
2. Currently, the feature extraction is somewhat poorly done, the next improvement to be done is to consider other efficient solutions to extract feature from the given input data
3. The architecture can be further carried over into transformers which may reduce compute and result in faster inferences. For example a branch can emerge from one of the transformer layers that specializes in quality of answer which may create a automatic feedback system for the LLM and allow scrutinizing it's own answers before answering.